
CI/CD Deployment Flow

□ Flow Name:

Code □ GitHub □ GitHub Actions □ Docker Image □ Deploy to EC2

□ Step-by-Step CI/CD Workflow:

1 □ Code Push to GitHub

-

Developer writes application code (Flask, Node.js, etc.)

- Code is pushed to a GitHub repository (e.g., main branch)

- Includes:

- Dockerfile
- App source code
- GitHub Actions workflow (.github/workflows/deploy.yml)

2□ Trigger GitHub Actions Workflow

- GitHub detects the push to main (or any configured branch)
 - GitHub Actions starts running the CI/CD pipeline defined in the workflow YAML file
-

3□ Build Docker Image

- GitHub Actions performs these CI steps:
 - Checks out the latest code
 - Logs into Docker Hub using GitHub Secrets
 -

Builds a Docker image from the Dockerfile

- Tags the image as your-dockerhub-username/your-image-name:latest

4□ Push Docker Image to DockerHub

- The built Docker image is pushed to DockerHub registry
- This makes the image accessible to your EC2 server

5□ Deploy to EC2

- GitHub Actions uses SSH to connect to your EC2 instance
- SSH credentials (private .pem key) are securely stored in GitHub Secrets
- Actions performed on EC2:
 - Pull the latest Docker image from DockerHub
 - Stop and remove any existing container (if running)
 - Start a new container using the new image
 - Expose it on desired port (e.g., port 80)

□ Final Outcome

- Application is successfully deployed and running inside a Docker container on the EC2 instance
- The app is accessible at:
`http://<EC2-Public-IP>`

□ GitHub Secrets Used

Secret Name	Description
DOCKER_USERNAME	Your Docker Hub username
DOCKER_PASSWORD	Docker Hub password or access token
EC2_PUBLIC_IP	Public IP of your EC2 instance

EC2_SSH_KEY	Contents of your .pem private key
-------------	-----------------------------------

□ Required Files

1.

Dockerfile □ for containerizing the app

2. docker-compose.yml file

3.

.github/workflows/deploy.yml □ GitHub Actions CI/CD

4.

App code (e.g. Flask app.py, Node index.js)

□ Example Use Case

Use Case: Deploy a Flask/Node.js app to EC2 every time you push to GitHub
No manual work needed after Git push.
